

Week 2 · PID Control Fundamentals

USV Guidance, Navigation and Control (Graduate) · Department of Autonomous Vehicle System Engineering, Chungnam National University | revised 2026-09-21

★ Reference material — read this first

The five courses below are **taught by the instructor of this course** and are the assumed background for it. They run from the fundamentals down to the graduate material, so start wherever the gap is. Every frame, symbol and derivation used here is developed in them at length; anyone whose prerequisites are thin should work through them first, then return.

#	Course	Level	Lang.	Video	Slides and code
1	Control Engineering (제어공학) — the foundation: transfer functions, feedback, stability, root locus, PID	undergraduate	KO	playlist	drive
2	Control System Design (제어시스템설계) — design rather than analysis: specifications, loop shaping, discrete implementation	undergraduate	KO	playlist	drive
3	Advanced Control Engineering (제어공학특론) — reference frames, the 6-DOF equation of motion, rotation matrices and Euler angles, linearisation and trim, vehicle control design	graduate	EN	playlist	drive
4	Sensor Signal Processing and Fusion (센서신호처리 및 융합) — sensor models, noise, estimation and multi-sensor fusion	graduate	EN	playlist	drive
5	Capstone Design (캡스톤디자인) — a vehicle project carried end to end	undergraduate	KO	playlist	drive

Not fluent in MATLAB or Simulink yet? Do these before Part 2. Every laboratory in this course is Simulink, and the Onramp courses are free and take a few hours each.

Tool	Where to start
MATLAB	MATLAB Onramp · Core MATLAB Skills
Simulink	Simulink Onramp · instructor's Simulink lectures part 1 · part 2

★ Key video references for this week

This week is built on the two video sources below as well as on the courses above. Watch them before or after the lecture hour; the sections that use each one say so where it is used.

Source	Part	What it contributes here
MATLAB Tech Talk, Understanding PID Control (B. Douglas, MathWorks) — playlist	1 What is PID control	present, past and future error — §2-2 to §2-5
	2 Anti-windup	windup, clamping and its two conditions — §2-8
	3 Noise filtering	the filtered derivative and its integrator-in-the-loop form — §2-7
	4 A PID tuning guide	requirements first, "well-behaved" plants, the map of tuning methods — §2-9
	5 Three ways to build a model	first principles, system identification, linearisation — §2-1
	6 Manual and automatic tuning	PID as two zeros and a pole; always look at the control effort — §2-6, §2-9
	7 Important PID concepts	cascaded loops and sampled control — §2-10
제어조교 Ctrl튜브, PID 제어기 째튜닝 (KO) — video	whole video	tune P first and read the plant from it, add I only for a remaining error, filter the derivative, smooth the setpoint — §2-7, §2-9

🔗 Getting the course files, and keeping them current (Windows)

The notes, models and scripts are kept in one Git repository that is updated through the semester. Clone it once; before every class, pull. A pull downloads only what has changed since the last one.

When	Where to run it	Command
once	PowerShell — installs Git for Windows	<code>winget install --id Git.Git -e</code>
once	the folder that will hold the course, e.g. <code>Documents</code>	<code>git clone https://github.com/wkyouncnu/Sensor-Signal-Processing-and-Fusion.git</code>
before every class	inside the cloned folder <code>Sensor-Signal-Processing-and-Fusion</code>	<code>git pull</code>

- `git pull` prints `Already up to date.` when nothing has changed, and otherwise lists the files it updated.
- The repository is private. When Git asks, sign in with the GitHub account the instructor has given access.
- Experiment on copies, not on the cloned files: copy a week's `WXX_simulink` folder elsewhere first, and a pull can never collide with local edits. If it already has, `git stash`, then `git pull`, then `git stash pop` sets the edits aside, updates, and puts them back.
- The MSS toolbox is not part of the repository. The weeks that simulate the Otter need it at `Tools\MSS` inside the cloned folder; this week does not.

- **Course:** USV Guidance, Navigation and Control (Graduate)

- **Department:** Autonomous Vehicle System Engineering, Chungnam National University
- **This week:** ① one error, read three ways — P, I and D, each measured on its own ② the three additions no real PID runs without: a filtered derivative, a smooth setpoint, anti-windup ③ a tuning order, each gain chosen from the measurement before it

★ Prerequisites from the previous week

- Week 1 built the Otter from first principles and ran it **open loop**: shaft speeds in, motion out, nothing measured and nothing corrected.
- This week closes a loop for the first time, but deliberately not around the vessel. The plant here is chosen so that nothing but the controller can explain a change in the response. Week 3 puts the same controller around the Otter.
- From basic control: the Laplace transform, a transfer function, and the step response of a second-order system. Course 1 in the table above covers all three.

Learning Outcomes

Upon completion of this week, the learner is able to:

1. Write the PID law in the time domain and as a transfer function, and state in one sentence what each of the three terms reads from the error.
2. Predict, before simulating, the steady value, damping ratio and overshoot of a proportional loop around a second-order plant, and confirm them to the third digit.
3. Explain why a proportional controller leaves an error, why the integral removes it, and derive the value of K_i at which the loop goes unstable.
4. Build a PID from Gain, Integrator and Transfer Fcn blocks that agrees with the Simulink PID block to round-off, and name the block that each field of the PID block's dialog belongs to.
5. Quantify what a pure derivative does to sensor noise, what a step setpoint does to the force, and what an integrator does while the actuator saturates — and apply the remedy for each.
6. Tune a PID in a fixed order, choosing every gain from the measurement before it, and check the result against the force the actuator can deliver.

Prerequisites and Setup

Item	Requirement
MATLAB	R2024b, with Simulink and the Control System Toolbox (<code>tf</code> , <code>step</code> , <code>c2d</code>)
MSS	not needed this week — the plant is a transfer function
Course folder	<code>lectures/W02_simulink</code>
Models	one per example — <code>W02_C_P</code> , <code>W02_D_PD</code> , <code>W02_E_PID</code> , <code>W02_F_block_vs_hand</code> , <code>W02_G_derivative_bench</code> , <code>W02_G_noise_kick</code> , <code>W02_H_antiwindup</code> , <code>W02_I_tuning</code> — all generated by <code>W02_1_build_pid</code> , never edited by hand
Time	lecture 1 h, laboratory 1 h (Part 2 sections A to J are done at home as well)

Part 1 · Theory

2-1. Why the first controller is not put around the vessel

This section answers one question: on what should a controller be learned?

- A vessel changes several things at once when one gain changes. Drag grows with speed, thrust follows the square of the shaft speed, and turning couples surge, sway and yaw. A change in the response cannot be attributed to the gain alone.
- A controller is therefore learned first on the **simplest plant that can oscillate**: one mass on one spring, with one damper. Anything that changes in its response is the controller's doing.

The plant is written from first principles — Newton's second law for a mass m held by a spring of stiffness k and a damper of coefficient b , pushed by a force τ :

$$m\ddot{y} + b\dot{y} + ky = \tau \quad \Rightarrow \quad G(s) = \frac{Y(s)}{T(s)} = \frac{1}{ms^2 + bs + k} = \frac{1}{s^2 + 2s + 2}$$

Symbol	Quantity	Value · source
y	position of the mass, the controlled output	m
τ	force on the mass, the control input	N
m	mass	1 kg, W02_0_setup.m
b	damping coefficient	2 N·s/m
k	spring stiffness	2 N/m
$\omega_{n0} = \sqrt{k/m}$	natural frequency without control	1.414 rad/s
$\zeta_0 = b/(2\sqrt{mk})$	damping ratio without control	0.707
$G(0) = 1/k$	static gain: metres per newton	0.5 m/N

- The poles are $s = -1 \pm j$: left alone the mass rings a little and stops. The plant is stable, nearly linear and has no delay — "well behaved" in the sense of MATLAB Tech Talk part 4, which is the class of plant for which the tuning rules of §2-9 are meant.
- The same plant is used by the Capstone Design course, week 6, section E, whose measured numbers this week reproduces.

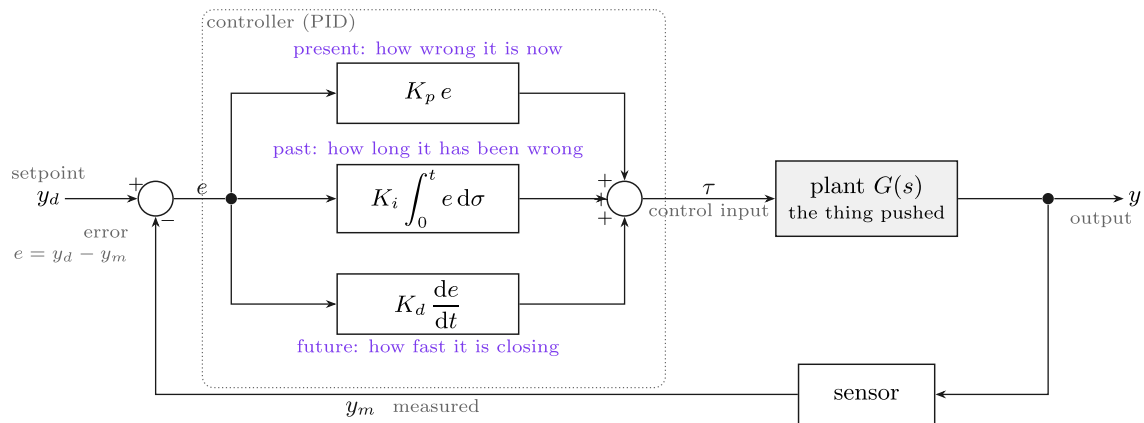
i Where a plant model comes from

MATLAB Tech Talk part 5 names three routes, and this course uses all three.

Route	What is known	Where in this course
first principles ("white box")	the physics, written as equations	this section; the Otter of Week 1, from Fossen's equations
system identification ("black box")	only input and output records	Week 3 §3-1 reduces the surge axis to two measured numbers
linearisation	a nonlinear model, about an operating point	Week 4, the yaw axis about straight running

2-2. Feedback, and one error read three ways

This section answers: what does a controller see, and what does it do with it?



In the figure	Meaning
y_d , setpoint	where the output should be — also called command or reference
circle with + and —	subtracts the measurement from the setpoint; its output is the error e
three boxes in the dotted frame	the controller: the same error read three ways, in violet above and below each box
τ , control input	the sum of the three, pushed into the plant
sensor, y_m	the measured output fed back; this week $y_m = y$ plus noise when noise is switched on

A controller receives **one** signal, the error, and the PID reads it three ways:

$$e(t) = y_d(t) - y_m(t), \quad \tau(t) = \underbrace{K_p e(t)}_{\text{present}} + \underbrace{K_i \int_0^t e(\sigma) d\sigma}_{\text{past}} + \underbrace{K_d \frac{de(t)}{dt}}_{\text{future}}$$

Term	What it reads from the error	In words (Tech Talk part 1)	What it cannot do alone
P, $K_p e$	how wrong it is now	"far away, push hard"	hold a position against a spring: zero error gives zero force
I, $K_i \int e$	how long it has been wrong	"still not there, push more"	act quickly: it responds late and stops late
D, $K_d \dot{e}$	how fast the error is closing	"about to arrive, brake"	know where the target is: a constant error gives no derivative

The picture to keep. On this plant each term does something a mechanic would recognise, and §2-3 to §2-5 prove it from the equation of motion:

Term	Acts like	So it
P	a spring pulling the mass towards y_d	makes the loop faster and bouncier; a spring alone never holds against another spring without stretching
D	a shock absorber (damper)	calms the bounce; it resists motion, not position
I	a patient hand that pushes harder the longer the error lasts	removes the error that the springs leave; being patient, it is also slow and can overdo it

Symbol	Quantity	Unit on this plant
K_p	proportional gain	N/m
K_i	integral gain	N/(m·s)
K_d	derivative gain	N·s/m

⚠ Why the textbook letters r and u are not used

Control textbooks write the setpoint r and the control input u . In this course u is the **surge velocity** and r the **yaw rate** — Week 3's speed loop is $u_d - u$ — so a page on which u is both a speed and a control input cannot be read. The setpoint is therefore y_d and the control input τ , as in Fossen's notation. y this week is a generic output, not the east position y^n .

2-3. P — the present error: faster, more oscillatory, never exact

This section answers: what does a proportional controller achieve on this plant, and what can it never achieve?

With $\tau = K_p(y_d - y)$ substituted into the equation of motion:

$$m\ddot{y} + b\dot{y} + (k + K_p)y = K_p y_d \quad \Rightarrow \quad T(s) = \frac{Y(s)}{Y_d(s)} = \frac{K_p}{ms^2 + bs + k + K_p}$$

- **The controller adds stiffness.** K_p sits beside k : a proportional controller is a second, adjustable spring pulling towards y_d .
- Reading the standard second-order form off the denominator,

$$\omega_n = \sqrt{\frac{k + K_p}{m}}, \quad \zeta = \frac{b}{2\sqrt{m(k + K_p)}}, \quad M_p = 100 \exp\left(\frac{-\pi\zeta}{\sqrt{1 - \zeta^2}}\right) \%$$

- The steady value follows from the final value theorem for a unit step, $y_{ss} = T(0)$:

$$y_{ss} = \frac{K_p}{k + K_p}, \quad e_{ss} = 1 - y_{ss} = \frac{k}{k + K_p}$$

Check	Result
dimensions	K_p and k are both N/m, so y_{ss} is dimensionless per metre of setpoint
limit $K_p \rightarrow 0$	$y_{ss} \rightarrow 0$: no controller, no motion
limit $K_p \rightarrow \infty$	$y_{ss} \rightarrow 1$ but $\zeta \rightarrow 0$: the error vanishes only as the damping does
numeric	verify_w02_pid check 2: formula against the step response of $T(s)$, largest difference 2.0×10^{-9}

Why the error never reaches zero. Holding the mass at y against the spring takes a steady force ky . A proportional controller produces force only from error, $\tau = K_p e$, so to produce ky some error must remain: $e_{ss} = ky_{ss}/K_p$, which rearranges to the formula above. Tech Talk part 1 makes the same argument with a drone that needs a fixed propeller speed to hover.

Measured in section C:

K_p	y_{ss}	M_p	e_{ss}	rise [s]	settle [s]	peak τ [N]
2	0.500	16.3 %	0.500	0.819	4.04	2.0
10	0.833	38.8 %	0.167	0.377	3.94	10.0
50	0.962	64.4 %	0.038	0.158	3.64	50.0

- Every measured y_{ss} and M_p equals the formula to the digits shown. $K_p = 2$ gives $\zeta = 0.5$ and $\omega_n = 2$ rad/s, the textbook's **16.3 %**.
- The last column is the price: at the step the whole error is 1 m and the force jumps to K_p newtons. A gain chosen from the response alone can ask the actuator for more than it has — Tech Talk part 6 shows a design that looked perfect until its 350 V command met a 24 V motor.

2-4. D — the future error: a brake

This section answers: what does the derivative term add, and why does it not remove the error?

With $\tau = K_p e + K_d \dot{e}$ and $e = y_d - y$ substituted into the equation of motion,

$$m \ddot{y} + (b + K_d) \dot{y} + (k + K_p) y = K_p y_d + K_d \dot{y}_d$$

$$\zeta = \frac{b + K_d}{2\sqrt{m(k + K_p)}}, \quad T(s) = \frac{K_d s + K_p}{ms^2 + (b + K_d)s + k + K_p}$$

- **K_d sits beside the damping b .** The derivative term is a second, adjustable damper: while the mass rushes towards the setpoint the error is shrinking, $\dot{e} < 0$, and the term takes force away before the setpoint is reached.
- The steady value is unchanged, $T(0) = K_p/(k + K_p)$. In steady state nothing moves, $\dot{e} = 0$, and the derivative contributes nothing.
- The numerator has gained a **zero** at $s = -K_p/K_d$. A zero closer to the origin than the poles lifts the early response, so the response can overshoot even when $\zeta > 1$.

i The same term lands elsewhere on other axes

On this plant K_d lands beside the damping, because the output is a position and its derivative a velocity. In Week 3 the output is a velocity, its derivative an acceleration, and the same term lands beside the **mass** (Week 3 §3-4). In Week 4 the output is an angle and K_d is a damper again. The term does not change; the axis does.

Measured in section D, $K_p = 10$, $K_i = 0$:

K_d	ζ	zero at	M_p	settle [s]	e_{ss}	peak τ [N]
0	0.289	none	38.8 %	3.94	0.167	10.0
2	0.577	−5.00	16.9 %	1.40	0.167	49.9
6	1.155	−1.67	8.6 %	1.27	0.167	129.6

- The overshoot and the settling time fall together; the error left does not move.
- At $K_d = 6$ the damping ratio exceeds one and the response still overshoots. With an ideal derivative the poles are real, at -2 and -6 , and the zero at -1.67 alone produces **5.55 %**; the model's filtered derivative (§2-7) gives the measured **8.6 %**.
- The peak force grows with K_d : at the corner of the step the filtered derivative jumps to $K_d N_f$ times the step height. That spike is the derivative kick of §2-7.

2-5. I — the past error: time removes it, and too much destabilises

This section answers: how does the integral remove the error that P and D leave, and what does it cost?

With all three terms, the Laplace transform of the loop gives the characteristic polynomial

$$ms^3 + (b + K_d)s^2 + (k + K_p)s + K_i = 0, \quad \frac{E(s)}{Y_d(s)} = \frac{s(ms^2 + bs + k)}{ms^3 + (b + K_d)s^2 + (k + K_p)s + K_i}$$

The error goes to zero. For a unit step, the final value theorem gives $e_{ss} = \lim_{s \rightarrow 0} s E(s) = 0$: the factor s in the numerator comes from the integrator. The loop is now **type 1**.

The integral finds the force on its own. The integral stops changing only when its input, the error, is zero. Where it stops is therefore exactly the force that holds the mass against the spring, $k y_d = 2 \text{ N}$ — a number the controller was never told.

Too much integral destabilises. The Routh array of the cubic above is stable if and only if every coefficient is positive and

$$(b + K_d)(k + K_p) > m K_i \quad \Rightarrow \quad K_i < \frac{(b + K_d)(k + K_p)}{m} = \frac{(2 + 4)(2 + 10)}{1} = 72$$

- At the limit the cubic has a pair of roots on the imaginary axis at $s = \pm j\sqrt{(k + K_p)/m} = \pm j3.464 \text{ rad/s}$ ([verify_w02_pid](#) check 4, exact to 10^{-9}).
- The controller actually built uses the filtered derivative of §2-7, which adds a pole; its limit, found by bisection on the closed-loop poles, is $K_i = 87.07$, oscillating at **3.856** rad/s. The limit that matters is the one of the implemented law.

Symbol	Quantity	Value
K_p, K_d	held fixed in section E	10 N/m, 4 N·s/m
$K_i^{\max}$, ideal derivative	Routh limit	72 N/(m·s)
$K_i^{\max}$, filtered, $N_f = 20$	pole-crossing limit	87.07 N/(m·s)

Measured in section E:

K_i	y at 10 s	M_p	settle [s]	error left	I at 10 s [N]
0	0.8333	9.8 %	1.32	1.67×10^{-1}	0.000
4	0.9957	0.0 %	4.84	4.33×10^{-3}	1.958
12	1.0000	4.7 %	1.49	3.39×10^{-6}	2.000

- $K_i = 4$ removes the error but slowly: the settling time grows from 1.32 s to 4.84 s. $K_i = 12$ removes it fast and brings the overshoot back.
- Run at **0.9** and **1.1** times the limit, the largest error between 30 and 40 s is **0.046** m and **13.353** m: the same oscillation decays on one side and grows on the other. The integral acts late, and a large enough late push arrives in phase with the swing it was meant to correct.

2-6. One transfer function: two zeros, a pole, and the Simulink block

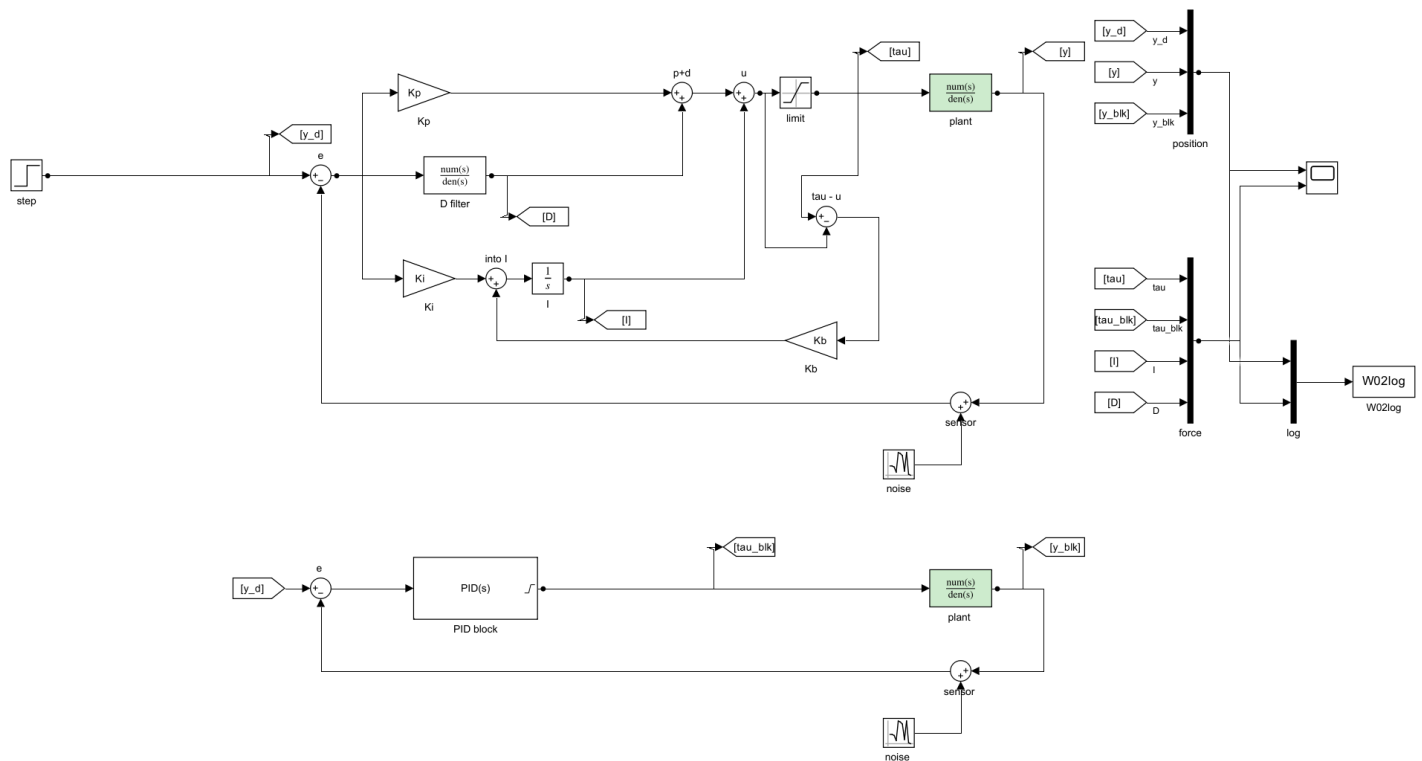
This section answers: what is tuning, seen as a transfer function, and is the Simulink PID block the same thing as the law above?

$$C(s) = K_p + \frac{K_i}{s} + K_d s = \frac{K_d s^2 + K_p s + K_i}{s}$$

- A PID is **one pole at the origin** (the integrator, fixed), **two zeros** whose places K_p , K_i and K_d decide, and an overall gain. Tuning a PID is choosing where those two zeros go and how much gain to apply (Tech Talk part 6).
- The implemented derivative is filtered (§2-7), which adds one more pole at $s = -N_f$:

$$C(s) = K_p + \frac{K_i}{s} + K_d \frac{N_f s}{s + N_f}$$

This is exactly the formula printed in the dialog of the Simulink **PID Controller** block (parallel form), with P, I, D and N its four fields. Model **w02_F_block_vs_hand** builds the same law from ordinary blocks on the top row and puts the library block on the bottom row, and runs both through a force limit and sensor noise.



THE LIBRARY PID BLOCK, anti-windup = back-calculation.
Same gains and limit (τ_{max}) as the row above, and its own noise source with the SAME seed, so both rows see the same noise. Its filter coefficient is $Nf_bik (= Nf)$.

WEEK 2, SECTION F - THE PID BLOCK AGAINST THE SAME LAW BY HAND

$e = y_d - y$, $\tau = K_p e + K_i \int e + K_d (N_f s / (s + N_f)) e$
 τ limited to $|\tau| \leq \tau_{max}$; $\dot{I} = K_i e + K_b (\tau - u)$
 y is measured with noise of standard deviation $noise_std$

Change a gain in the Command Window (e.g. $K_p = 20$) and press Run:
the Scope shows the position (top) and the force (bottom) at once.

In the figure	Meaning
K_p , D filter, $K_i + I$	the three terms, one block each; D filter is the transfer function $K_d N_f s / (s + N_f)$
$p+d$, u	the sums that form the demand $u = P + D + I$
limit	the actuator: $\tau = \text{sat}(u)$, $ \tau \leq \tau_{max}$
$\tau - u$, K_b and the line back to into I	the back-calculation of §2-8
PID block on the bottom row	the library block with the same gains, limit and K_b
Goto tags and the Scope on the right	every signal reaches one Scope: position on top, force below

Measured in section F, with $|\tau| \leq 2.5$ N and 5 mm of sensor noise:

Comparison	largest $ \Delta y $ [m]	largest $ \Delta \tau $ [N]
built from blocks against the PID block	9.99×10^{-16}	6.35×10^{-14}
control: the block's N_f changed from 20 to 19	1.20×10^{-3}	0.187

- The two rows agree to round-off through the limit, the anti-windup and the noise.
- The control row shows the comparison is not blind: a 5 % change in one number shows at once.

Field of the PID block dialog	Block of the law built by hand
Proportional (P)	Kp
Integral (I)	Ki , before the integrator
Derivative (D), Filter coefficient (N)	D filter , numerator [Kd*Nf 0] , denominator [1 Nf]
Back-calculation coefficient (Kb)	Kb

2-7. The derivative in practice: noise, and the corner of a step

This section answers: why is a textbook derivative never implemented as written, and what replaces it?

A derivative amplifies by speed, not by size. For one frequency component of any signal,

$$\frac{d}{dt} A \sin(\omega t) = A \omega \cos(\omega t)$$

- The amplitude is multiplied by ω . Sensor noise is the fastest signal present, so the derivative selects the noise and amplifies it without bound (Tech Talk part 3).

The remedy is a first-order low-pass filter in front of the derivative:

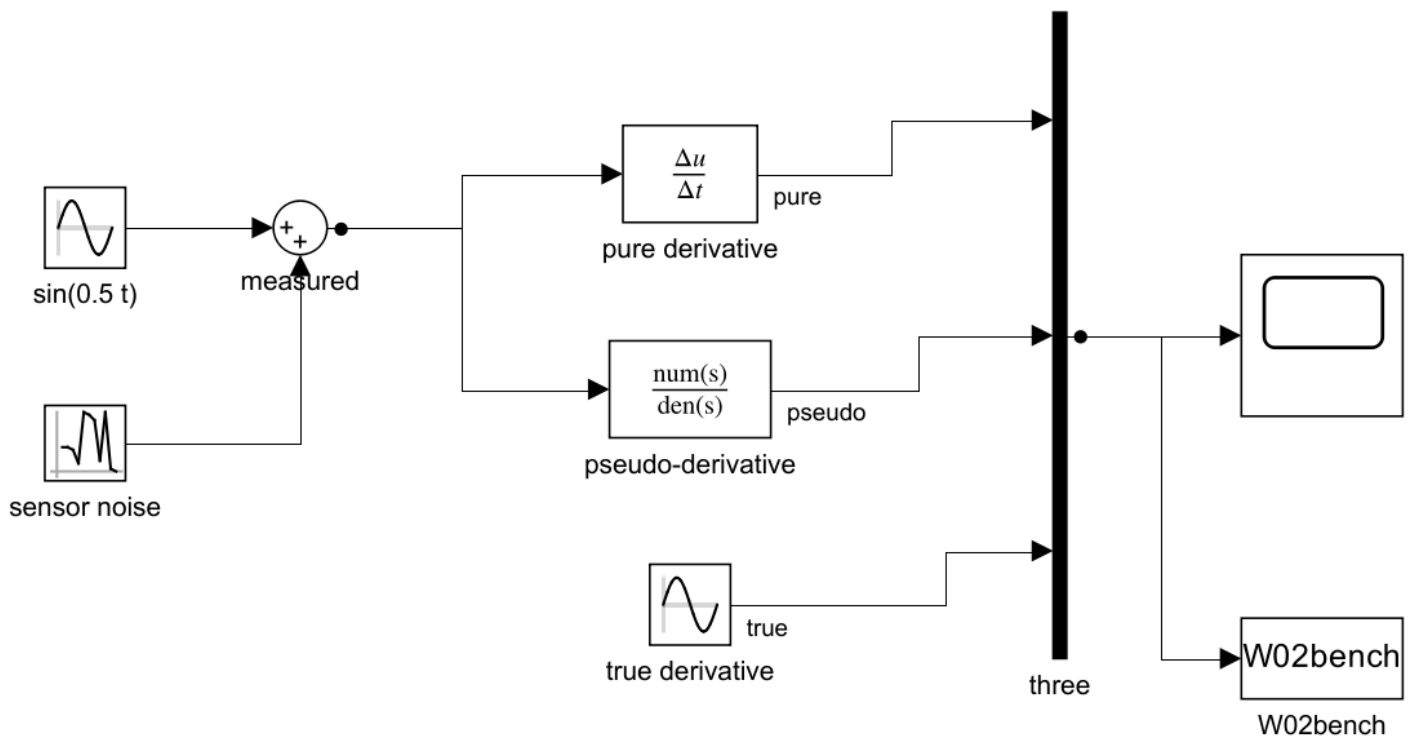
$$D(s) = K_d \frac{N_f s}{s + N_f} \qquad |D(j\omega)| \approx \begin{cases} K_d \omega, & \omega \ll N_f \quad (\text{a derivative}) \\ K_d N_f, & \omega \gg N_f \quad (\text{a ceiling}) \end{cases}$$

Symbol	Quantity	Value
N_f	filter coefficient: where differentiating stops	20 rad/s; "Filter coefficient (N)" in the PID block, written N in Week 3
$K_d N_f$	the ceiling on the derivative's gain	80 N·s/m at $K_d = 4$

- `verify_w02_pid` check 5: $|D(j\,0.01)|/K_d = 0.01000$ and $|D(j\,10^5)|/K_d = 20.000$.

In Simulink this is one **Transfer Fcn** block, numerator `[Kd*Nf 0]` and denominator `[1 Nf]` — the block `D filter` in every model of this week. Tech Talk part 3 shows the same thing built as a gain N_f with an integrator in its feedback path, $N_f/(1 + N_f/s)$; the two are one transfer function (`verify_w02_pid` check 5).

Seeing it on a bench. Model `w02_G_derivative_bench` has no loop at all. It adds noise of standard deviation 5 mm to $\sin(0.5t)$, whose true derivative $0.5 \cos(0.5t)$ never exceeds 0.5, and differentiates the result twice — once purely, once through the filter.



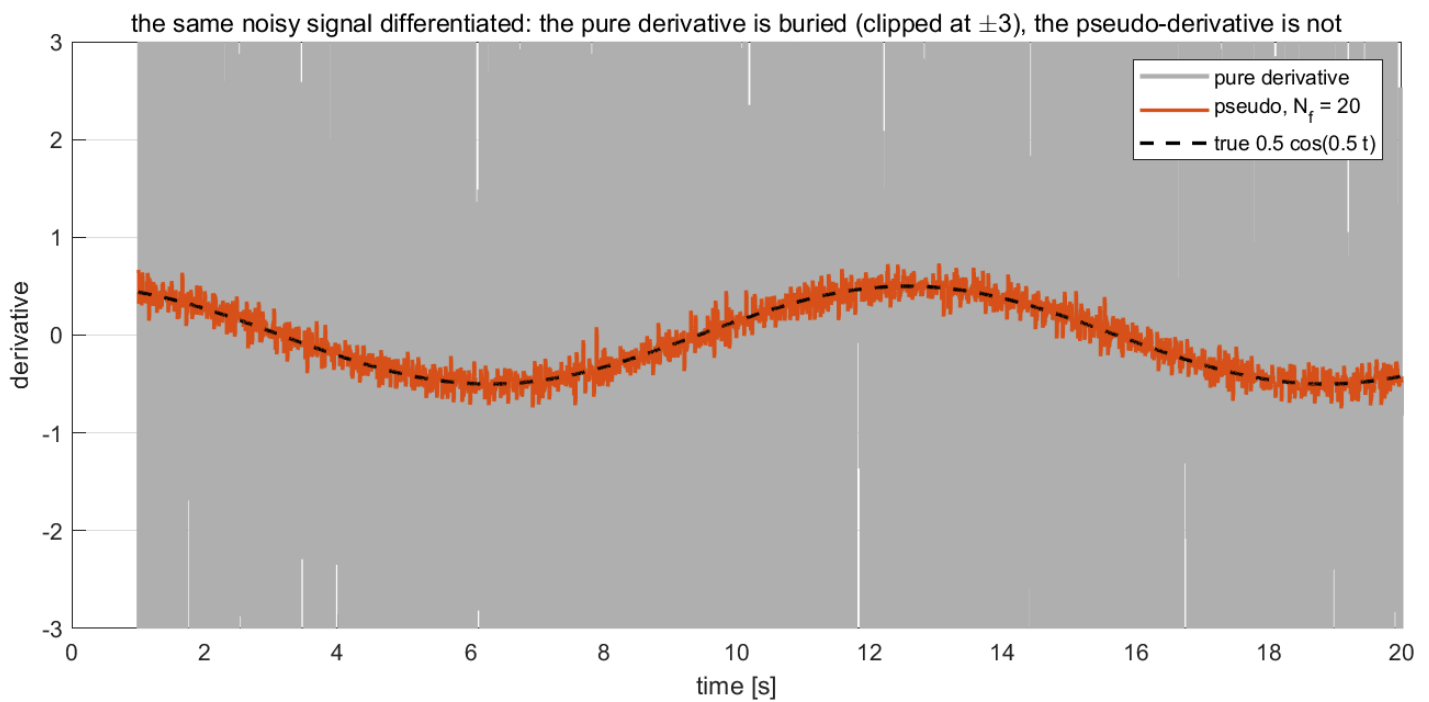
WEEK 2, SECTION G - THE DERIVATIVE BENCH (no loop)

$\text{measured} = \sin(0.5 t) + \text{noise of standard deviation bench_noise}$
 $\text{pure} = d/dt (\text{measured})$ the textbook derivative
 $\text{pseudo} = N_f s / (s + N_f) (\text{measured})$ the filtered derivative
 $\text{true} = 0.5 \cos(0.5 t)$ what both should show

The noise is tiny, yet the pure derivative is buried under spikes.
 Change N_f in the Command Window (5, 20, 200) and press Run.

In the figure	Meaning
$\sin(0.5 t) + \text{sensor noise} \rightarrow \text{measured}$	the signal a sensor would report
pure derivative	the textbook d/dt
pseudo-derivative	$N_f s / (s + N_f)$, the same filter as D filter with $K_d = 1$
true derivative	$0.5 \cos(0.5 t)$, what both should show

N_f	largest pure derivative	largest pseudo-derivative	true
5	26.55	0.56	0.5
20	26.55	0.75	0.5
200	26.55	5.18	0.5



In the figure	Meaning
grey	the pure derivative, clipped at ± 3 so that the rest can be seen; it reaches 26.55
orange	the pseudo-derivative with $N_f = 20$
black dashed	the true derivative $0.5 \cos(0.5t)$

- Noise of 5 mm turns the pure derivative into spikes 53 times larger than the true value. The pseudo-derivative follows the truth, and the higher its ceiling N_f , the more noise it lets through.

The corner of a step. A step setpoint makes the error jump. The filtered derivative of a jump of height Δ starts at $K_d N_f \Delta$ and decays with time constant $1/N_f$ — the **derivative kick** ([verify_w02_pid](#) check 6: **80.000 N** for $K_d = 4$, $N_f = 20$, $\Delta = 1$). An unfiltered derivative of a step is an impulse. The 제어조교 Ctrl튜브 video's remedy is to take the corner off the setpoint by passing it through a first-order filter $1/(T_f s + 1)$, so that its slope is at most Δ/T_f .

i Two other remedies, used later

Differentiating the measurement instead of the error — setting the setpoint weight $c_d = 0$ — removes the kick altogether, and is the choice Week 3 §3-4 and Week 4 make. A reference model that produces a smooth setpoint together with its derivative is the complete answer; it is the subject of Week 8.

In the loop ([w02_g_noise_kick](#) , $K_p = 10$, $K_i = 8$, $K_d = 4$), measured in section G:

Derivative	ceiling $K_d N_f$	force std [N], $t > 6$ s	y std [mm], $t > 6$ s	M_p , no noise
$N_f = 5$	20	0.16	0.59	16.0 %
$N_f = 20$	80	0.45	0.57	0.0 %
$N_f = 200$	800	3.06	0.54	0.0 %
pure derivative	none	9.29	0.60	9.0 %

Setpoint	peak force [N]	peak D term [N]
step	89.7	79.7
smoothed, $T_f = 0.3$ s	10.5	8.1

- The force chatters in proportion to the ceiling, while the position wanders by **0.54** to **0.60** mm in all four runs. The extra force is spent on the noise, not on the mass.
- Too low a ceiling costs damping instead: at $N_f = 5$ the filter delays the derivative so much that it no longer brakes in time, and the overshoot returns to **16 %**. N_f sits above the frequencies of the response and below those of the noise.
- Smoothing the setpoint cuts the peak force by a factor of **8.5**; the response pays only by arriving as late as the smoothed setpoint does.

2-8. The integral in practice: the actuator has a limit

This section answers: what goes wrong when the actuator saturates, and how is the integrator protected?

- Every real actuator saturates: a motor has a maximum speed, a thruster a maximum thrust. In a linear model any force is available; in hardware it is not (Tech Talk part 2).
- While the actuator sits on its limit, the error stays large, and the integrator — which does not know about the limit — keeps accumulating. When the output finally reaches the setpoint, the stored charge keeps pushing, and it can be removed only by **negative** error, that is, by overshooting. This is **integrator windup**.

Two remedies are in common use:

Scheme	What the integrator does while saturated	Source
clamping (conditional integration)	stops integrating when both the output is saturated and the error has the same sign as the output — the integral would only make it worse	Tech Talk part 2
back-calculation	integrates $K_i e + K_b(\tau - u)$: the excess the actuator cut off is fed back and pulls the integrator back	Åström & Murray, Ch. 11

For back-calculation, the integrator's input is

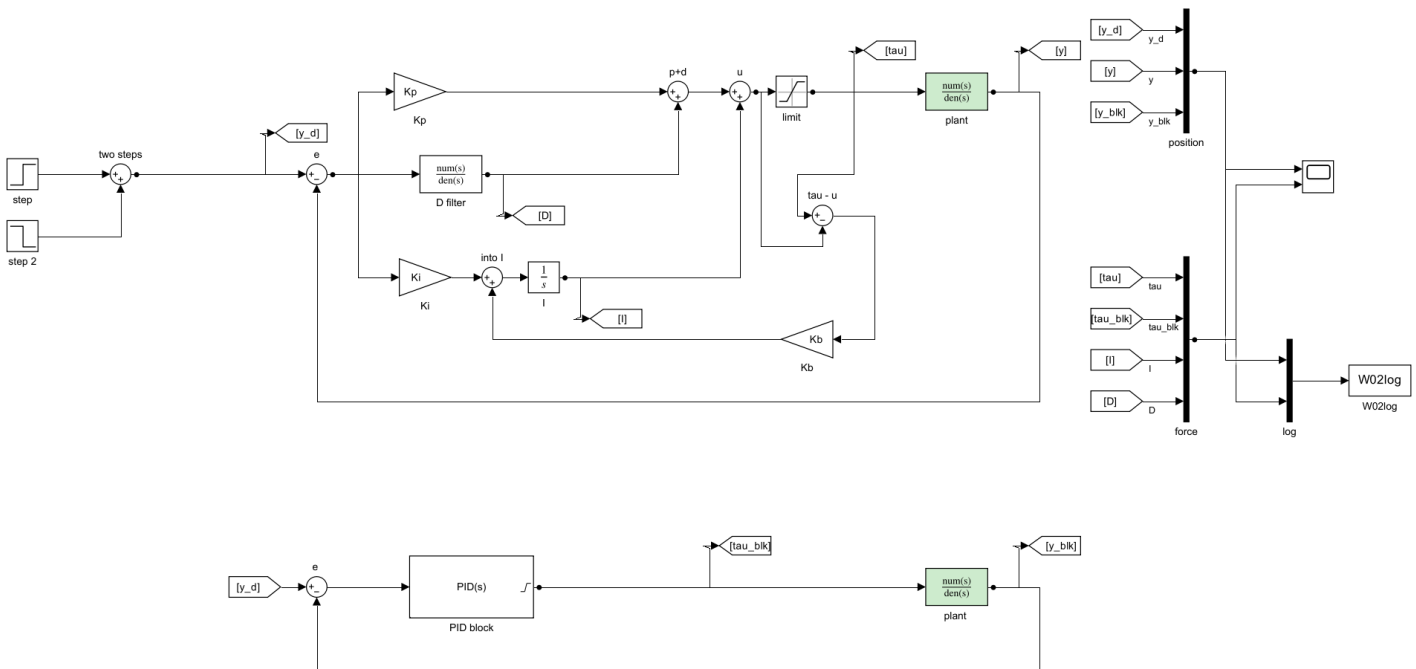
$$\dot{I} = K_i e + K_b(\tau - u), \quad u = P + I + D, \quad \tau = \text{sat}(u)$$

and while the actuator is saturated at $\tau_{\max}$, setting $\dot{I} = 0$ shows where the integrator is driven:

$$I^* = \tau_{\max} - P - D + \frac{K_i}{K_b} e$$

- I^* is the value that makes the demand equal to the limit. When P alone already asks for more than $\tau_{\max}$, I^* is **negative**: the integrator goes below zero, and that is the formula working, not a fault.
- Tech Talk part 2 adds a practical rule: set the controller's limit a little below the actuator's physical one, because the physical limit drifts with temperature and wear.
- Week 3 §3-6 derives all three schemes on the Otter and compares them with the Simulink block.

The experiment that makes windup obvious. Model `w02_H_antiwindup` limits the force to $|\tau| \leq 2.5$ N. Against a spring of $k = 2$ N/m that can hold the mass at most at $2.5/2 = 1.25$ m. The target is set to an **unreachable** 2 m from $t = 1$ to 15 s, then lowered to a reachable 0.5 m. The top row is the law built from blocks with back-calculation ($K_b = 0$ turns it off); the bottom row is the library PID block, whose anti-windup method the script switches between none, clamping and back-calculation.



THE LIBRARY PID BLOCK, anti-windup = clamping.
Same gains and limit (tau_max) as the row above, and its own noise source with the SAME seed, so both rows see the same noise. Its filter coefficient is NF_bik (= NF).

WEEK 2, SECTION H - A FORCE LIMIT, AND ANTI-WINDUP
$$e = y_d - y, \quad \tau = K_p e + K_i \int e + K_d (Nf s / (s + Nf)) e$$
$$\tau \text{ limited to } |\tau| \leq \tau_{\max}; \quad \dot{I} = K_i e + K_b (\tau - u)$$

Change a gain in the Command Window (e.g. $K_p = 20$) and press Run:
the Scope shows the position (top) and the force (bottom) at once.

In the figure	Meaning
step , step 2 , two steps	the setpoint: 2 m at $t = 1$ s, 0.5 m at $t = 15$ s
limit , tau - u , Kb	the force limit and the back-calculation path into into I
PID block	the library block, anti-windup method set per run by the script

Measured in section H, $K_p = 10$, $K_i = 8$, $K_d = 4$:

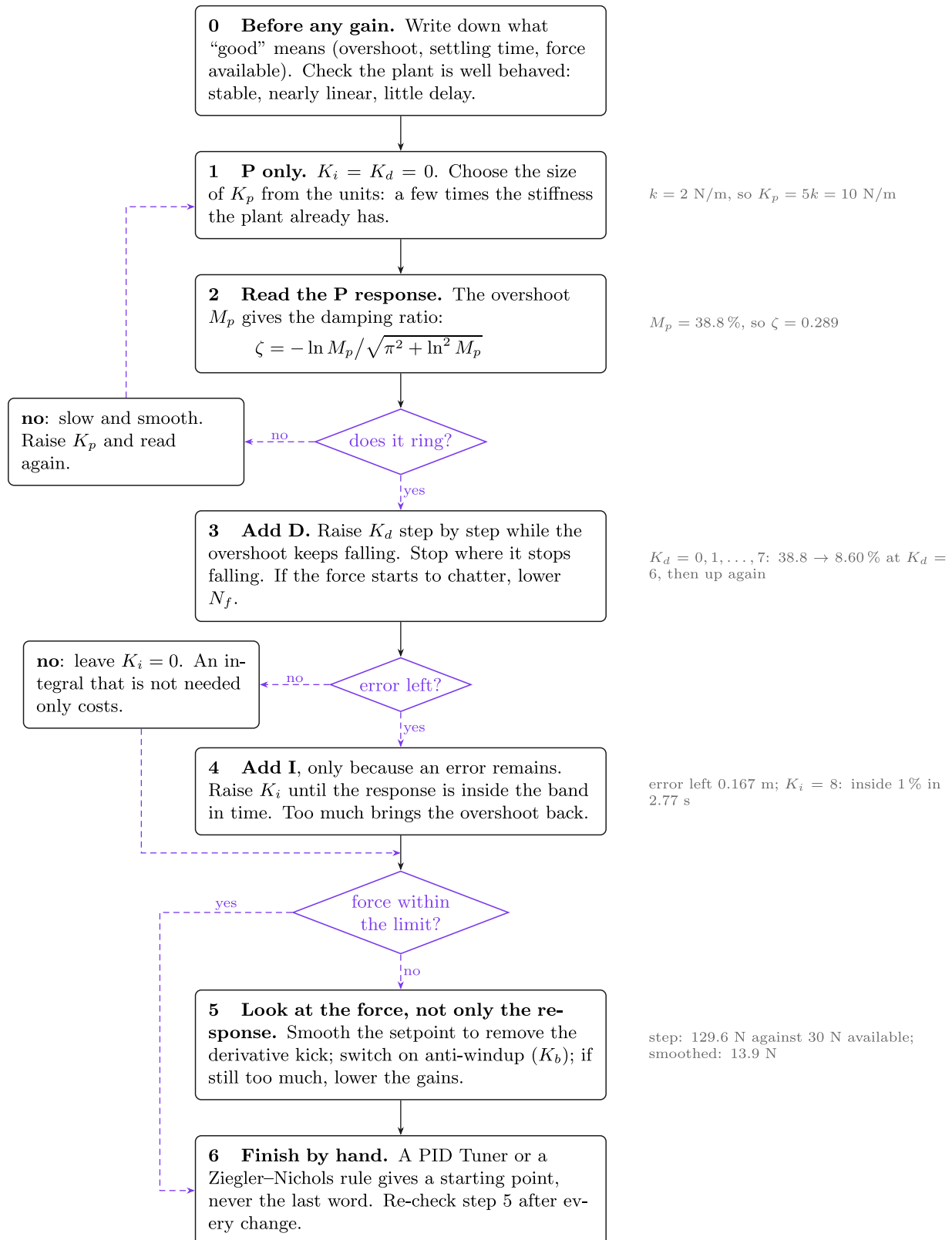
Anti-windup	y at 15 s [m]	back within 2 % of 0.5 m after [s]
none	1.250	never (within the 15 s that remain)
clamping	1.250	4.41
back-calculation	1.250	4.06

- Without anti-windup the integrator has stored **94.0 N** by $t = 15$ s — the actuator can give 2.5 N. When the target drops, that charge keeps the force on its limit for the rest of the run: the mass does not come back. On a vessel this looks like ignoring a stop command.
- With back-calculation the integrator sits at **-1.96 N** at $t = 15$ s. The formula predicts it: $e = 2 - 1.25 = 0.75$, $P = 7.5$ N, $D = 0$, so $I^* = 2.5 - 7.5 + (8/2)(0.75) = -2.0$ N.
- Clamping and back-calculation both recover in about 4 s; back-calculation also reproduces the library block to 6.66×10^{-16} m.
- The back-calculation gain matters little here: $K_b = 0.5, 2, 10, 50$ recover in 4.26, 4.06, 4.37 and 4.28 s.

2-9. The tuning order — one gain at a time

This section answers: in what order are the three gains chosen, and from what?

Tuning a PID by hand: one gain at a time, each chosen from the last measurement



In the figure	Meaning
numbered boxes, top to bottom	the steps, done in this order and one at a time
violet diamonds	the three questions that decide the next step: does it ring, is an error left, is the force within the limit
violet dashed arrows	the branches: back to K_p , skip the integral, skip the force fix
grey text on the right	what each step measured on this plant in section I

Step	What is done	Why in this order
0	write down what "good" means — overshoot, settling time, the force available — and check that the plant is well behaved	without a requirement there is no point at which to stop; for a plant that is unstable, strongly nonlinear or delayed these rules may not apply (Tech Talk part 4)
1	P only, $K_i = K_d = 0$; choose the scale of K_p from the units	the error is in metres and the force in newtons: K_p must be several times the stiffness the plant already has, here $k = 2$ N/m, so $K_p = 5k$ (Ctrl+투브)
2	read the P response: the overshoot gives the damping ratio	$\zeta = -\ln M_p / \sqrt{\pi^2 + \ln^2 M_p}$; ringing means the next term is D, not I (Ctrl+투브; verify_w02_pid check 3 confirms the formula is the exact inverse)
3	raise K_d while the overshoot keeps falling; stop where it stops falling	the derivative has an optimum: its zero at $-K_p/K_d$ moves towards the origin and pushes the response up again (§2-4); if the force chatters, lower N_f
4	add K_i only if an error remains, until the response is inside the band in time	an integral that is not needed only costs — slower settling, returning overshoot, windup (§2-5)
5	look at the force , not only the response	smooth the setpoint, switch on anti-windup, or lower the gains (Tech Talk part 6)
6	finish by hand	a PID Tuner or a Ziegler–Nichols rule gives a starting point, never the last word (Tech Talk parts 4 and 6); repeat step 5 after every change

Applied to this plant in section I, with the targets "inside 1 % within 3 s" and "30 N available":

Stage	K_p	K_d	K_i	M_p	inside 1 % after	peak force
1–2 P only	10	0	0	38.77 %	never	10.0 N
3 P + D	10	6	0	8.60 %	never	129.6 N
4 P + I + D	10	6	8	0.16 %	2.77 s	129.6 N
5 the same, setpoint smoothed	10	6	8	0.15 %	3.11 s	13.9 N

- Step 2 read $M_p = 38.8\%$ as $\zeta = 0.289$, the value $b/(2\sqrt{k + K_p})$ predicts.
- Step 3 raised K_d through $0, 1, \dots, 7$; the overshoot fell $38.77 \rightarrow 24.95 \rightarrow 16.94 \rightarrow 12.34 \rightarrow 9.83 \rightarrow 8.71 \rightarrow 8.60$ and rose to 9.22 at $K_d = 7$, so $K_d = 6$.
- Step 4 found $K_i = 8$ as the first integer gain inside 1 % within 3 s.
- Step 5 found the answer to the question the response never asks: the step demands **129.6 N** against **30 N** available. Smoothing the setpoint brings it to **13.9 N** at the cost of **0.34 s**.

2-10. Two ideas the rest of the course leans on

This section answers: what changes when the controller runs on a computer, and when one loop sits inside another?

Sampled control. A digital controller reads the sensor, computes, and holds its output until the next sample (Tech Talk part 7). While the sample time T_s is short against the closed loop's speed, nothing changes; past that, the controller acts on stale information.

Rule	Statement	Source
sample fast enough	sample at least 20 times the closed-loop bandwidth: $T_s \omega_B \leq 2\pi/20 = 0.31$	Franklin, Powell & Workman, <i>Digital Control of Dynamic Systems</i> , Ch. 11
tune in continuous time	design in s , convert to z , then check the sample time	Tech Talk part 7

Measured in section J, the default gains, $\omega_B = 6.11$ rad/s:

T_s [s]	$T_s \omega_B$	largest $ z $	M_p	settle [s]
continuous	—	—	0.0 %	2.06
0.01	0.06	0.9868	0.2 %	2.05
0.05	0.31	0.9364	1.4 %	2.00
0.1	0.61	0.8779	5.0 %	1.90
0.2	1.22	0.7734	25.6 %	1.80
0.3	1.83	0.6830	37.9 %	2.70
0.4	2.44	0.8105	67.8 %	7.20

- The same gains that are fine in continuous time go unstable at $T_s = 0.511$ s, where a closed-loop pole reaches the unit circle (`verify_w02_pid` check 7 finds **0.5115** s on a fine grid).
- This course simulates its controllers in continuous time, as MSS does. The table is the check to make before the gains go onto a computer running at a given rate.

Cascaded loops. When one loop's output is the setpoint of another, the inner loop can be tuned first and the outer loop designed as if the inner one were instantaneous — provided the inner loop is roughly **5 to 7 times faster** in bandwidth (Tech Talk part 7). The benefit is that the inner loop rejects its own fast disturbances before the outer loop sees them. Week 5 is the first cascade in this course: the guidance law is the outer loop and the heading autopilot of Week 4 the inner one.

Part 2 • Laboratory

Every example has **its own model**, and every model looks the same: the loop on the left, built from ordinary blocks on the top level, and **one Scope** on the right with the position on top and the force below. Open a model, press **Run**, and the Scope shows the response. Change a gain in the Command Window and press Run again — nothing else is needed.

Section	Model	What it shows
C	<code>W02_C_P</code>	P only
D	<code>W02_D_PD</code>	P + D
E	<code>W02_E_PID</code>	P + I + D
F	<code>W02_F_block_vs_hand</code>	the same law built from blocks and as the library PID block
G	<code>W02_G_derivative_bench</code> , <code>W02_G_noise_kick</code>	the pseudo-derivative on a bench, then in the loop with noise
H	<code>W02_H_antiwindup</code>	an unreachable target, and three anti-windup schemes
I	<code>W02_I_tuning</code>	the tuning order of §2-9

The section scripts only repeat what Run shows, for several gains at once, and print the numbers quoted in Part 1.

A. Setting up (5 min)

```
cd lectures/W02_simulink
W02_0_setup
```

Expected output:

```
W02_0_setup
plant    G(s) = 1/(1 s^2 + 2 s + 2)
gains    Kp = 10   Ki = 8   Kd = 4   Nf = 20
step     y_d = 0 -> 1 at t = 1 s
limit    |tau| <= 1e+06, Kb = 2
noise    std 0 m every 0.01 s
```

- `W02_0_setup.m` is the only file edited by hand. Every block holds a variable name from it.
- $\tau_{\max} = 10^6$ N means "no limit"; the models of F and H are run with **2.5** N.

⚠ **The first line of `W02_0_setup` is `clear`**

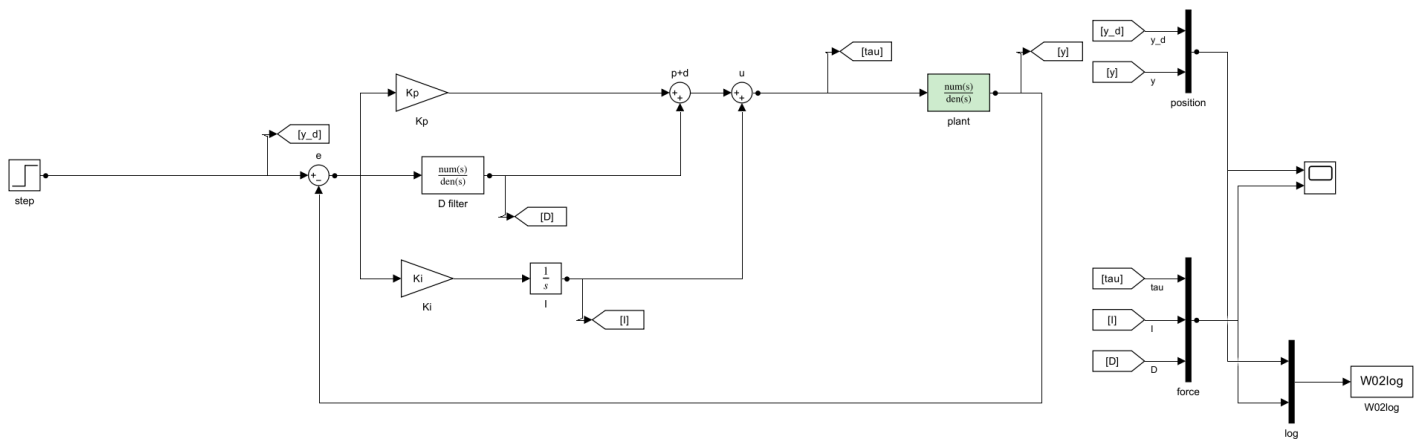
It resets the workspace to the lecture's values. Run it again whenever a Scope does not match these notes.

B. The models (10 min)

```
W02_1_build_pid
open_system('W02_E_PID')
```

Expected output ends with:

```
built W02_I_tuning.slx      (overlapping lines: 0)
built W02_G_derivative_bench.slx (overlapping lines: 0)
```



WEEK 2, SECTION E - P + I + D

$$e = y_d - y, \quad \tau = K_p e + K_i \int e \, dt + K_d (Nf s / (s + Nf)) e$$

Change a gain in the Command Window (e.g. $K_p = 20$) and press Run: the Scope shows the position (top) and the force (bottom) at once.

In the figure	Meaning
<code>step</code> → <code>e</code>	the setpoint and the error $e = y_d - y$
<code>Kp</code> , <code>D filter</code> , <code>Ki</code> → <code>I</code>	the three terms, one block each
<code>p+d</code> , <code>u</code> → <code>plant</code>	their sum is the force τ on the mass-spring-damper
the long line along the bottom	the feedback: the position back to <code>e</code>
Goto tags <code>[y_d]</code> , <code>[y]</code> , <code>[tau]</code> , <code>[I]</code> , <code>[D]</code>	carry the signals to the right without crossing the loop
<code>position</code> , <code>force</code> → Scope	one Scope, two panels: y_d and y on top; τ , I and D below

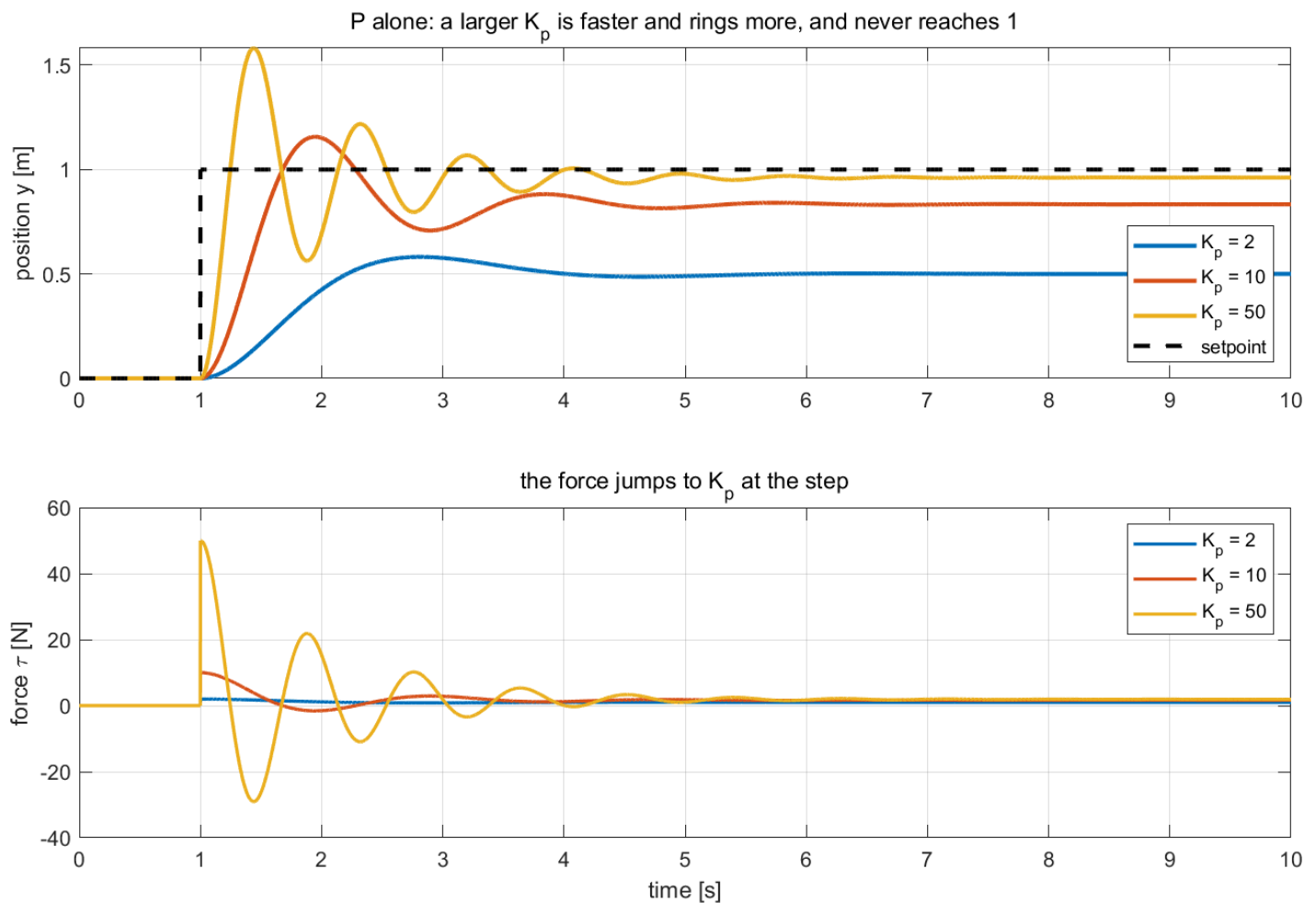
- Try it: `Kp = 30` , Run — the top panel rings more. `Ki = 0` , Run — the response settles short of 1.

C. P only (10 min)

W02_C_proportional_only

Expected output:

Kp	y _{ss} (formula)	overshoot % (formula)	rise [s]	settle [s]	peak tau [N]
2	0.500 (0.500)	16.3 (16.3)	0.819	4.04	2.0
10	0.833 (0.833)	38.8 (38.8)	0.377	3.94	10.0
50	0.962 (0.962)	64.4 (64.4)	0.158	3.64	50.0



In the figure	Meaning
top	the position for $K_p = 2, 10, 50$ and the dashed setpoint
bottom	the force; it jumps to K_p newtons at the step

What the figure says

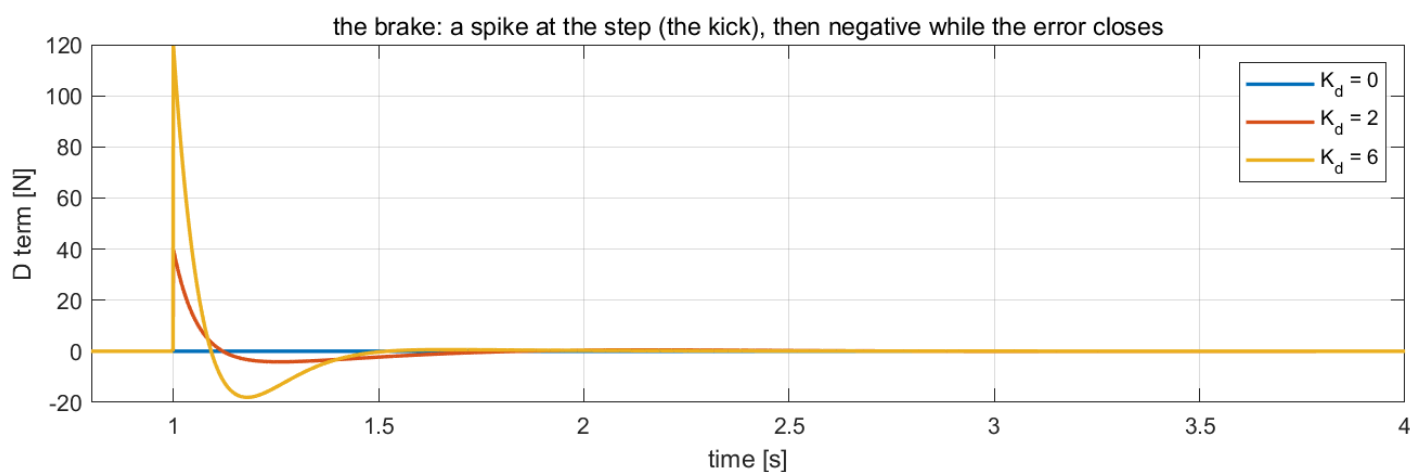
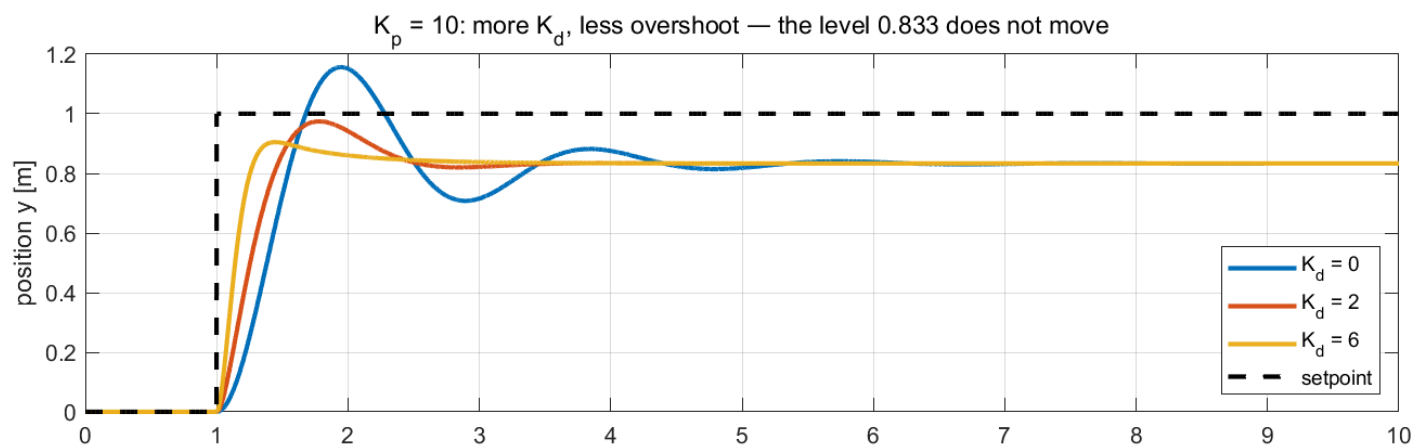
- Each larger gain is faster and rings more, and each settles below 1 at $K_p/(k + K_p)$. Measured and formula columns agree to the digits shown.

D. Add D (10 min)

W02_D_derivative

Expected output:

Kd	zeta	zero	overshoot %	settle [s]	error left	peak tau [N]
0	0.289	none	38.8	3.94	0.167	10.0
2	0.577	-5.00	16.9	1.40	0.167	49.9
6	1.155	-1.67	8.6	1.27	0.167	129.6



In the figure	Meaning
top	the position for $K_d = 0, 2, 6$ at $K_p = 10$
bottom	the derivative term D : a spike at the step, then negative while the error closes

What the figure says

- The shock absorber calms the bounce — overshoot $38.8 \rightarrow 8.6\%$ — and every curve still settles at **0.833**: D cannot remove the error.

E. Add I, and find its limit (15 min)

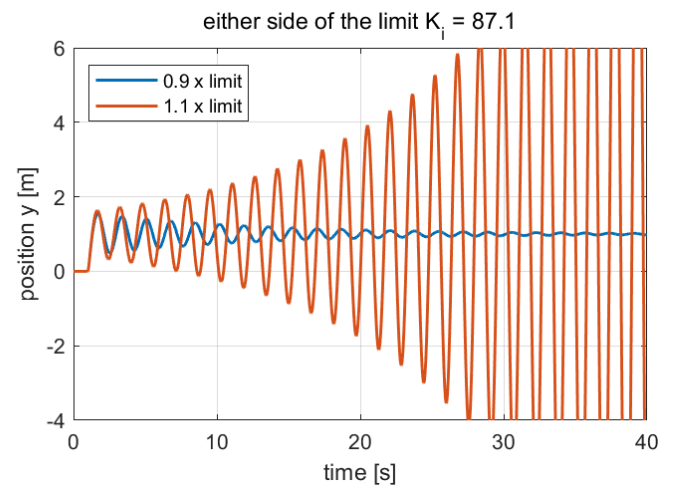
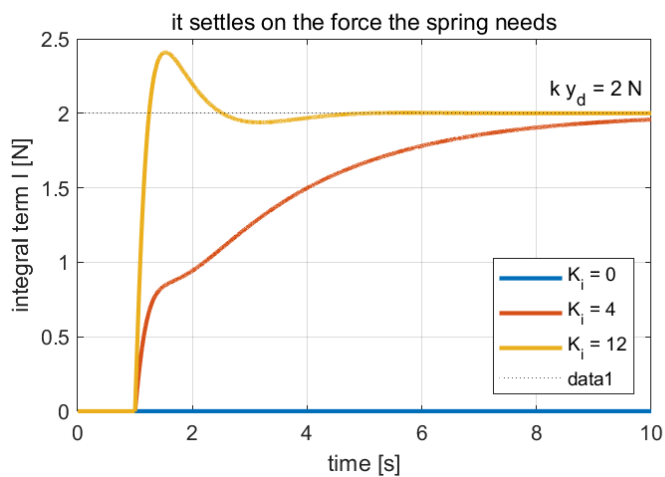
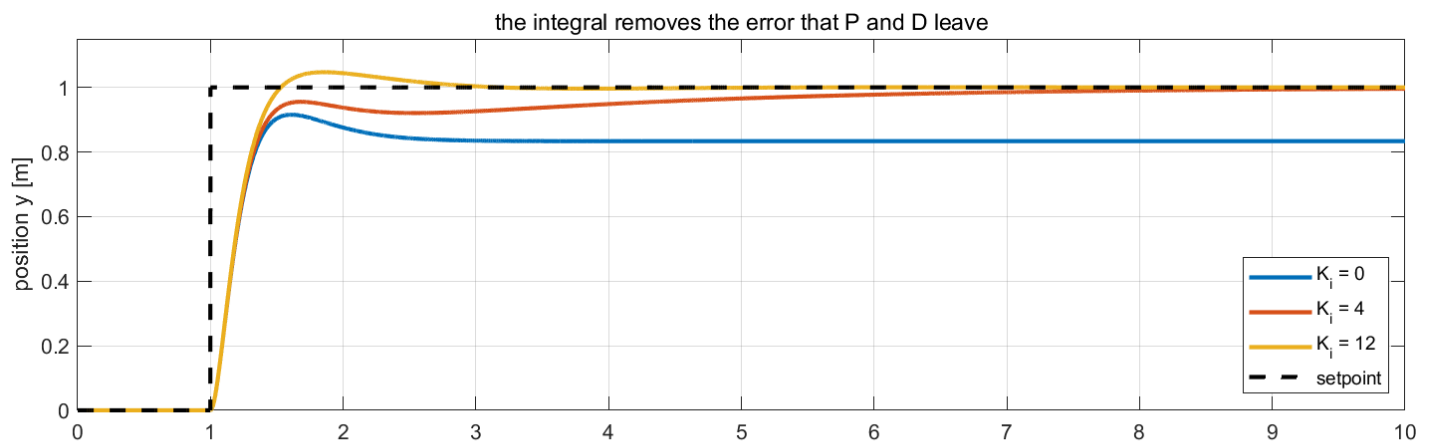
w02_E_integral

Expected output:

```

Ki    y at 10 s    overshoot %    settle [s]    error left    I at 10 s [N]
0      0.8333      9.8          1.32         1.67e-01      0.000
4      0.9957      0.0          4.84         4.33e-03      1.958
12     1.0000      4.7          1.49         3.39e-06      2.000
Routh, ideal derivative:  $K_i < (b + K_d)(k + K_p) = 72$ 
the filtered law of the model is unstable above  $K_i = 87.07$ 
  Ki = 0.9 x 87.07    0.046 m
  Ki = 1.1 x 87.07   13.353 m

```



In the figure	Meaning
top	the position for $K_i = 0, 4, 12$ at $K_p = 10, K_d = 4$
bottom left	the integral term; the dotted line is $ky_d = 2 \text{ N}$
bottom right	40 s at 0.9 and 1.1 times the stability limit

What the figure says

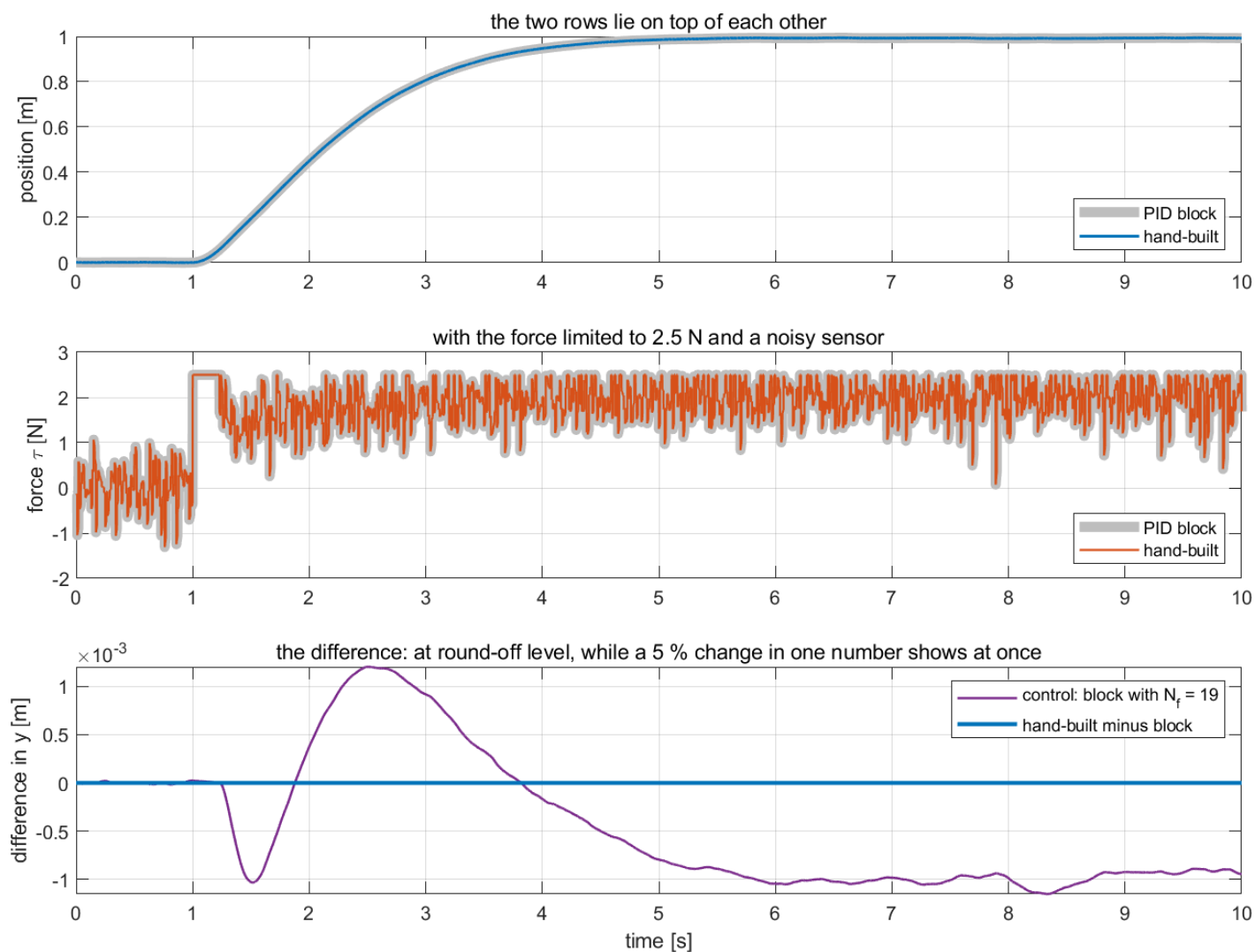
- The patient hand keeps pushing until the error is gone, and stops at exactly the **2 N** the spring needs. Pushed too hard, it makes the loop grow instead of settle.

F. The law built from blocks against the Simulink PID block (10 min)

w02_F_block_vs_handbuilt

Expected output:

comparison	max y diff	max tau diff
hand-built vs PID block	9.99e-16	6.35e-14
control: block with Nf = 19	1.20e-03	1.87e-01



In the figure	Meaning
top and middle	thick grey: the PID block; thin: the law built from blocks — with a 2.5 N limit and 5 mm of noise
bottom	the difference, at round-off, and the control run in violet

What the figure says

- The two lines cannot be told apart. The PID block's dialog is nothing more than the blocks of §2-6.

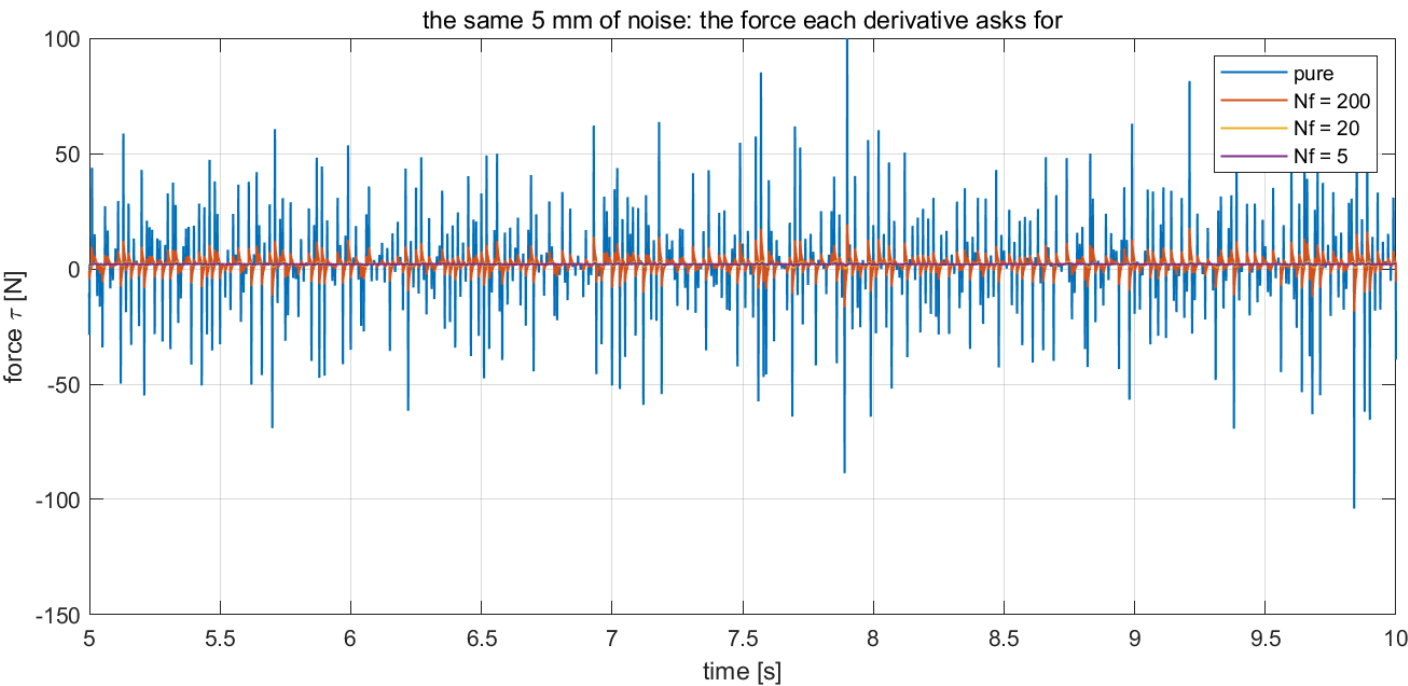
G. The pseudo-derivative (15 min)

```
open_system('W02_G_derivative_bench')      % press Run, then try Nf = 5 and Nf = 200
W02_G_derivative_noise_and_kick
```

Expected output:

Nf	largest pure	largest pseudo	
5	26.55	0.56	
20	26.55	0.75	
200	26.55	5.18	
derivative	force std [N]	y std [mm]	overshoot % (no noise)
pure	9.29	0.60	9.0
Nf = 20	0.45	0.57	0.0
Nf = 5	0.16	0.59	16.0
setpoint	peak force [N]	peak D [N]	
step	89.7	79.7	
smoothed	10.5	8.1	

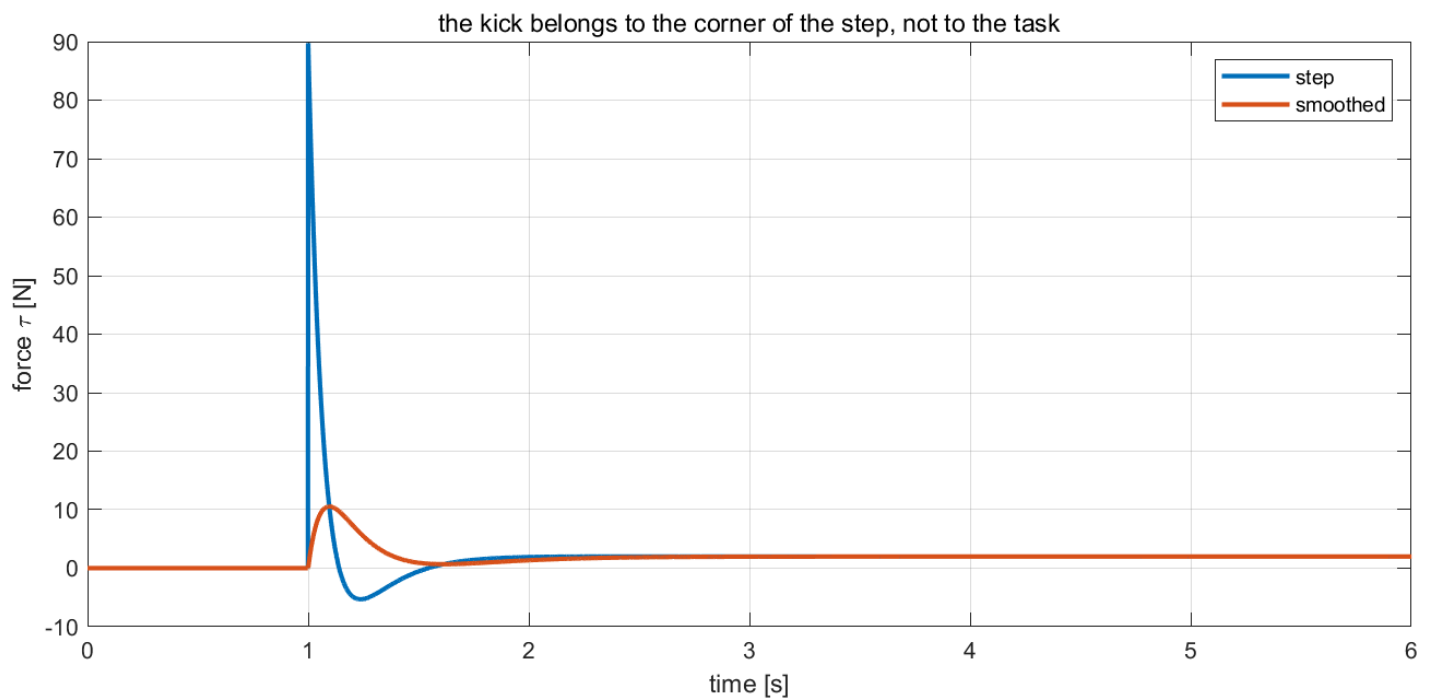
The bench figure is in §2-7.



In the figure	Meaning
traces	5 s of force with 5 mm of sensor noise: pure derivative, $N_f = 200, 20, 5$

What the figure says

- The pure derivative asks for tens of newtons to hold a mass that needs **2** N. The filter is what makes a derivative usable at all.



In the figure	Meaning
traces	the force for a step setpoint and for the same step smoothed by $1/(0.3s + 1)$

What the figure says

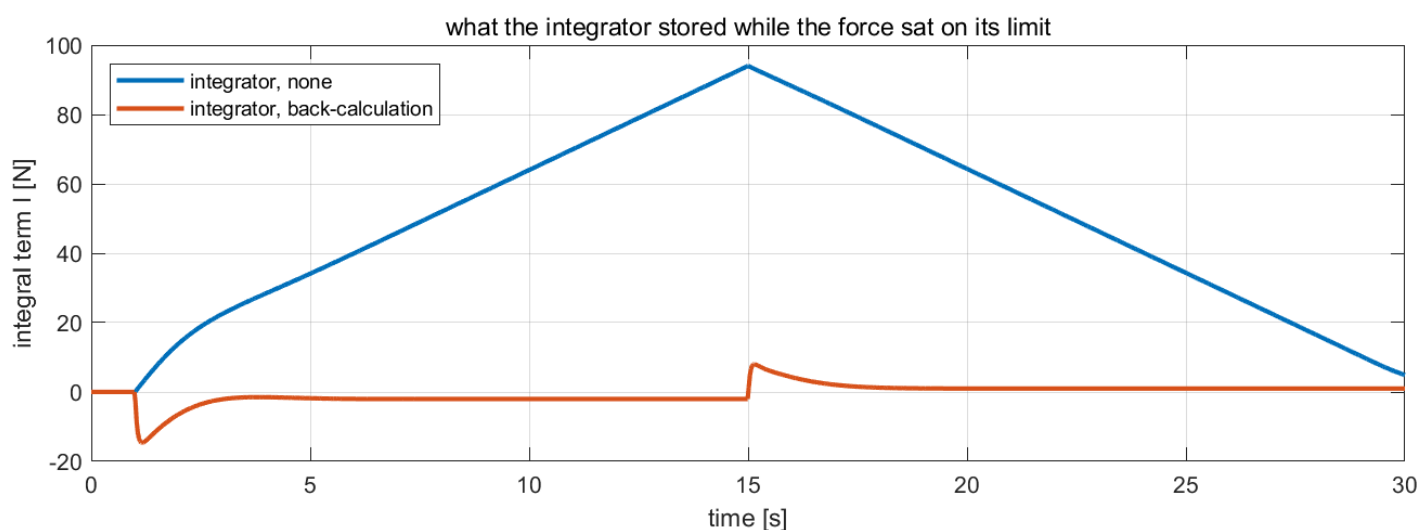
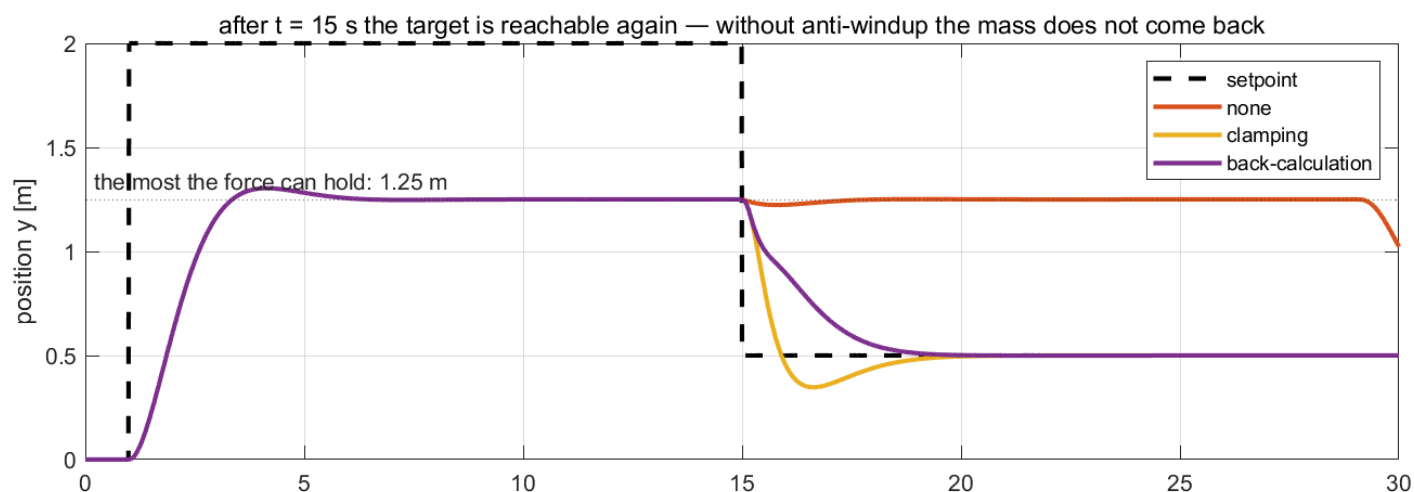
- 90 N at the corner of the step, 10.5 N without the corner. The kick belongs to the setpoint, not to the task.

H. Anti-windup, in depth (15 min)

```
open_system('W02_H_antiwindup')           % press Run with y_step = 2; t_step2 = 15; T_final = 30; ta
u_max = 2.5
W02_H_windup
```

Expected output:

```
none (block)           1.250           Inf
clamping (block)       1.250           4.41
back-calculation (block) 1.250           4.06
by hand vs block, back-calculation: max |y diff| = 6.66e-16
integrator at t = 15 s: 94.0 N without anti-windup, -1.96 N with back-calculation
```



In the figure	Meaning
top	the position for the three schemes; the dotted line is 1.25 m, the most the limited force can hold
bottom	the integrator without anti-windup and with back-calculation

What the figure says

- From 1 to 15 s every scheme sits at 1.25 m: the target cannot be reached. What differs is the memory. Without anti-windup the integrator has piled up 94 N, and after 15 s the mass stays where it is instead of going to 0.5 m. Clamping and back-calculation return in about 4 s.

I. The tuning order, step by step (15 min)

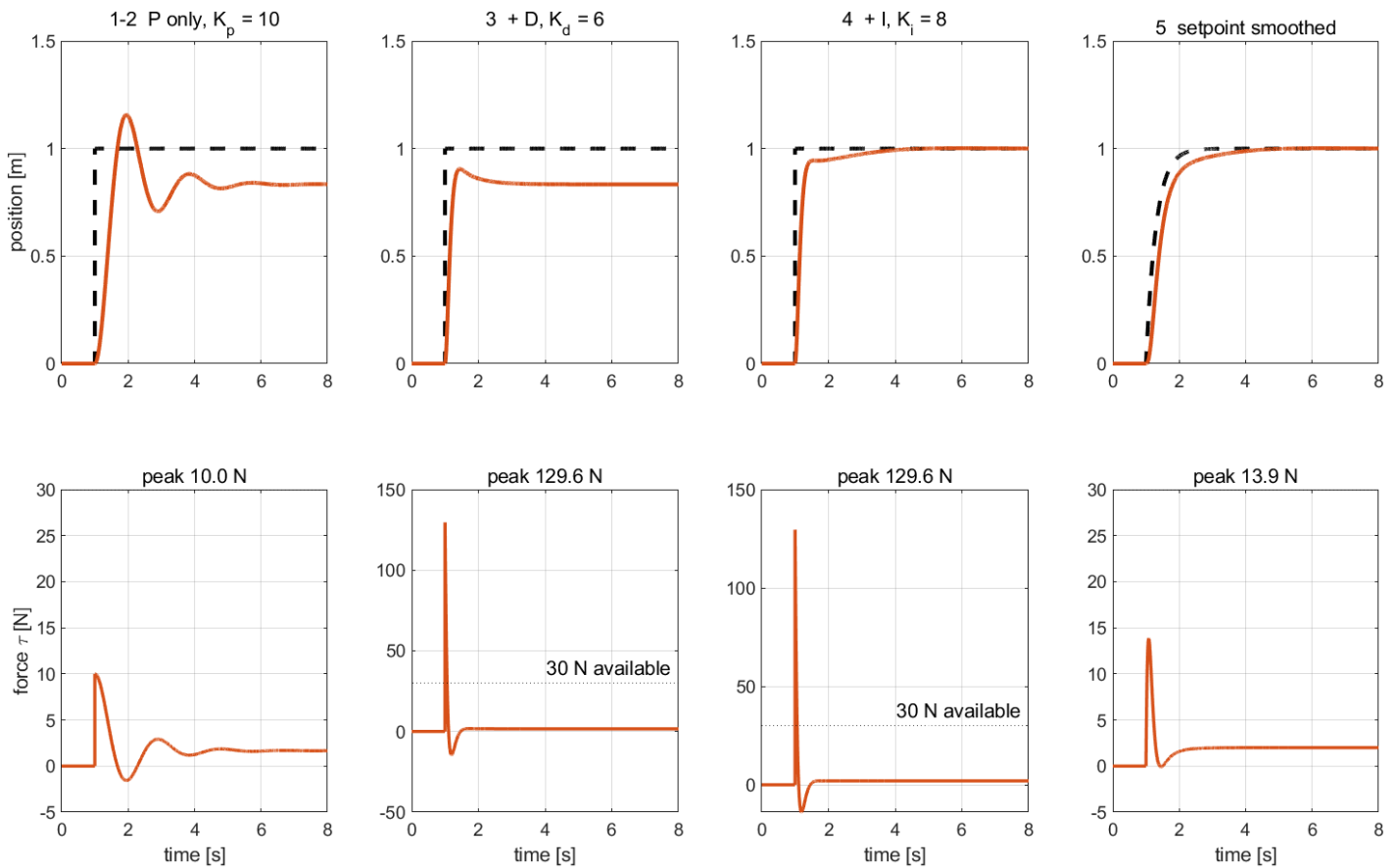
W02_I_tuning_by_hand

Expected output:

```

1-2 Kp = 10: overshoot 38.8 % -> damping ratio 0.289 (formula 0.289): it rings, so D next
3   Kd = 0,1,...: overshoot 38.77 24.95 16.94 12.34 9.83 8.71 8.60 9.22 % -> stops falling at K
d = 6
4   error left 0.167 -> Ki = 8: inside 1 % after 2.77 s, overshoot 0.16 %
5   peak force 129.6 N against 30 N available -> smoothed setpoint: 13.9 N, inside 1 % after 3.11
s

```



In the figure	Meaning
columns 1 to 4	P only, + D, + I, and the same with a smoothed setpoint
top row / bottom row	position / force against the 30 N available

What the figure says

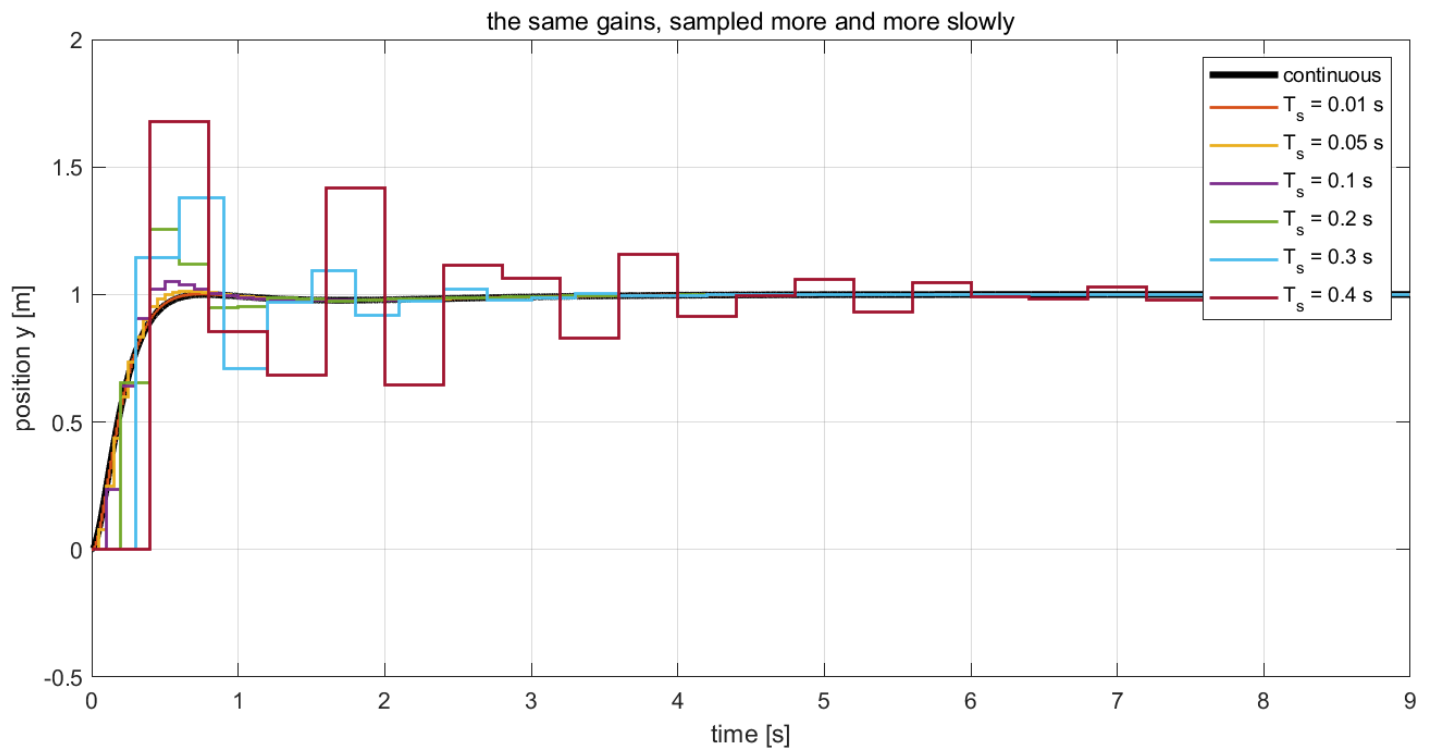
- Each column fixes what the one before it showed: P revealed ringing, D calmed it, I removed the error, and only the bottom row reveals that stage 4 asks for four times the available force.

J. The same PID on a computer (10 min)

W02_J_sample_time

Expected output:

Ts [s]	Ts*wB	overshoot %	settle [s]
cont.	-	0.0	2.06
0.01	0.06	0.2	2.05
0.05	0.31	1.4	2.00
0.1	0.61	5.0	1.90
0.2	1.22	25.6	1.80
0.3	1.83	37.9	2.70
0.4	2.44	67.8	7.20
unstable from Ts = 0.511 s (Ts*wB = 3.12)			



In the figure	Meaning
black	the continuous-time response
staircases	the sampled responses; each step is one held sample

What the figure says

- Up to $T_s \omega_B = 0.31$ the staircases lie on the black curve. Beyond it the controller acts on older and older errors, and at $T_s = 0.511$ s the loop goes unstable.

Summary

Week Summary

Step	What was done	How it was verified
1	wrote the plant from first principles, $G(s) = 1/(s^2 + 2s + 2)$	poles $-1 \pm j$; static gain 0.5 m/N
2	P as a spring: $y_{ss} = K_p/(k + K_p)$	section C: formula and measurement agree to three digits; <code>verify_w02_pid</code> check 2
3	D as a shock absorber, and its zero	section D: overshoot 38.8 $\rightarrow$ 8.6 %, error left unchanged at 0.167
4	I removes the error; Routh limit $K_i < 72$	<code>verify_w02_pid</code> check 4; section E: decay at 0.9 $\times$, growth at 1.1 $\times$ the implemented limit 87.07
5	the law from blocks equals the Simulink PID block	section F: difference 10^{-15} under a limit and noise; control run 1.2×10^{-3} m
6	the pseudo-derivative	section G: bench 26.55 against 0.75 (true 0.5); force std 9.29 $\rightarrow$ 0.45 N; kick 89.7 $\rightarrow$ 10.5 N
7	anti-windup against an unreachable target	section H: none never returns (94 N stored); clamping 4.41 s; back-calculation 4.06 s
8	followed the tuning order	section I: $K_p = 10$, $K_d = 6$, $K_i = 8$, each from the measurement before it
9	sampled the controller	section J: unstable at $T_s = 0.511$ s; <code>verify_w02_pid</code> check 7

Progress Check

★ Minimum condition for following Week 3

Theory

- ☐ Able to say what P, I and D each do on this plant in mechanical terms: spring, shock absorber, patient hand.
- ☐ Able to derive $y_{ss} = K_p/(k + K_p)$ and explain why it is never 1.
- ☐ Able to derive the Routh limit on K_i for a PID around a second-order plant.
- ☐ Able to explain the ceiling $K_d N_f$, the derivative kick, and I^* of back-calculation.

Laboratory

- ☐ `w02_1_build_pid` ran and every model reported `overlapping lines: 0`.
- ☐ A gain was changed in the Command Window and the model's Scope showed the new response.
- ☐ `w02_check(1)`, `(2)` and `(3)` pass on a model built from `w02_P1_start`.

Recorded observations

- ☐ The measured overshoot at $K_p = 2$, and the formula's.
- ☐ The K_d at which the overshoot stopped falling in section I.
- ☐ How long each anti-windup scheme took to come back in section H.

Assignment 2

- **Due:** before the Week 3 session
- **Submit:** the model, the script that produces every number, and the analysis

① Requirements

Change the plant to $G(s) = 1/(s^2 + 0.5s + 4)$ — a stiffer spring with far less damping — in `w02_0_setup.m` and `w02_vars.m` ($b = 0.5$, $k = 4$) and follow the tuning order of §2-9 with the targets: inside 2 % of a unit step within 4 s, overshoot below 10 %, peak force below 40 N.

② Verification — mandatory

★ A claim is not a result. Numbers are required, with the averaging window stated

- For step 2, the damping ratio read from the P response next to the value $b/(2\sqrt{m(k + K_p)})$.
- For step 3, the table of K_d against overshoot, and the K_d chosen.
- For step 4, the K_i chosen and the resulting settling time, measured with `step_metrics`.
- For step 5, the peak force with and without the smoothed setpoint.
- The Routh limit on K_i for the ideal derivative, and the limit of the filtered law found as in section E.

③ Analysis (5–10 lines)

Compare the gains with those of this plant. Explain, from where K_p and K_d land in the equation of motion, why the lightly damped plant needs a larger share of its damping from the controller.

Grading

Criterion	Weight
The model runs and produces the requested output	25%
Verification performed and numbers reported	40%
Correctness of the analysis	25%
Readability of the code	10%

Troubleshooting

- Only faults that have actually occurred while preparing this week.

Symptom	Cause	Fix
a script called with <code>run(...)</code> stops inside another function, or "brace indexing is not supported"	a script left a variable named <code>run</code> or <code>s</code> in the workspace, hiding the function of the same name (section E defines <code>s = tf('s')</code>)	run <code>w02_0_setup</code> again, or call the scripts one at a time
<code>w02_check(2)</code> reports <code>PID blocks 1</code> and fails	the PID Controller block was used; problem 2 asks for the law built from blocks	build D as one Transfer Fcn, <code>[Kd*Nf 0]</code> over <code>[1 Nf]</code>
the force in the Scope is a band of spikes of tens of newtons	<code>d_filtered = 0</code> in <code>w02_G_noise_kick</code> selects the pure derivative	<code>d_filtered = 1</code>
a Scope does not match these notes after experimenting	a changed variable is still in the workspace	run <code>w02_0_setup</code> again

References

Primary

- Åström, K. J. and Murray, R. M. *Feedback Systems: An Introduction for Scientists and Engineers*, 2nd ed., Princeton University Press, 2021, Ch. 11 (PID control: basic functions, tuning, integrator windup, implementation).
- Åström, K. J. and Häggglund, T. *Advanced PID Control*, ISA, 2006, Ch. 3 (setpoint weighting, derivative filtering, windup).
- Franklin, G. F., Powell, J. D. and Emami-Naeini, A. *Feedback Control of Dynamic Systems*, 8th ed., Pearson, 2019, §4.3 (the three-term controller).
- Franklin, G. F., Powell, J. D. and Workman, M. L. *Digital Control of Dynamic Systems*, 3rd ed., Addison-Wesley, 1998, Ch. 11 (sample rate selection).

Video

- Douglas, B. *Understanding PID Control*, parts 1–7, MATLAB Tech Talk, MathWorks, 2018 — [playlist](#).
- 제어조교 Ctrl튜브, [제어공학] PID 제어기 째튜닝 — Simulink 시뮬레이션, 2022 — [video](#).

In this course

- Capstone Design, week 6, sections E–G — the same plant and the undergraduate version of sections C to H.
- The instructor's *Linear Control Systems*, chapter 13, Practical PID Controller Design using MATLAB and Simulink.
- Week 3 §3-4 (where the derivative lands on a velocity loop) and §3-6 (three anti-windup schemes on the Otter).
- [_tools/verify_w02_pid.m](#) — the eight checks quoted in Part 1.

Next Week

- **Week 3 — Surge Speed Control**
- The first closed loop on the vessel. The surge equation of Week 1 §1-11 becomes the plant, the controller of this week is placed around it, and the steady speed is predicted before it is measured.
- The derivative term turns out to land beside the mass rather than the damping, and the thruster limit makes the windup of §2-8 real.
- Preparation: $T_u = 1.1025$ s and $K_u = 0.012894$ (m/s)/N from Week 1 §1-11, and the MSS toolbox at [Tools\MSS](#).